

Prepared by: Christine Khayat

Date: September 7, 2026

System Assessed: Hydroficient MQTT Pipeline (Development Environment)

Part 1: Reconnaissance

Traffic Interception

I ran my pipeline with three terminals (broker, publisher, subscriber), then opened a fourth terminal and ran:

Shell

```
mosquitto_sub -t "#" -v
```

Results after 30 seconds of capture:

- Messages captured: 15 messages
- Topics observed:
 - hydroficient/grandmarina/sensors/main-building/readings
- Information visible in each message:
 - Device ID: GM-HYDROLOGIC-01
 - Location: main-building
 - Timestamp (ISO 8601 UTC)
 - Counter / sequence number
 - Upstream pressure
 - Downstream pressure
 - Flow rate

Screenshot of Intercepted Traffic

None

```
mosquitto_sub -h localhost -p 1883 -t "#" -v
hydroficient/grandmarina/sensors/main-building/readings
{"device_id": "GM-HYDROLOGIC-01", "location": "main-building",
"timestamp": "2026-09-08T17:39:57.376847+00:00", "counter": 14,
"pressure_upstream": 83.4, "pressure_downstream": 76.3,
"flow_rate": 39.0}
hydroficient/grandmarina/sensors/main-building/readings
{"device_id": "GM-HYDROLOGIC-01", "location": "main-building",
"timestamp": "2026-09-08T17:39:59.378490+00:00", "counter": 15,
"pressure_upstream": 83.2, "pressure_downstream": 78.0,
"flow_rate": 39.8}
```

```
hydroficient/grandmarina/sensors/main-building/readings
{"device_id": "GM-HYDROLOGIC-01", "location": "main-building",
"timestamp": "2026-09-08T17:40:01.384128+00:00", "counter": 16,
"pressure_upstream": 81.7, "pressure_downstream": 74.3,
"flow_rate": 38.5}
hydroficient/grandmarina/sensors/main-building/readings
{"device_id": "GM-HYDROLOGIC-01", "location": "main-building",
"timestamp": "2026-09-08T17:40:03.384748+00:00", "counter": 17,
"pressure_upstream": 81.6, "pressure_downstream": 77.6,
"flow_rate": 37.4}
```

The screenshot displays a terminal window and a web dashboard. The terminal window shows the output of a Python script (sensor_publisher.py) and the Mosquitto MQTT broker logs. The web dashboard shows the received sensor data in a structured format.

Terminal Window (Hydroficient-project — Python sensor_publisher.py — 80x24):

```
[113] Pressure: 81.1/77.9 PSI, Flow: 42.3 gal/min
[114] Pressure: 80.6/76.4 PSI, Flow: 42.1 gal/min
[115] Pressure: 82.8/77.0 PSI, Flow: 41.2 gal/min
[116] Pressure: 83.9/77.7 PSI, Flow: 40.6 gal/min
[117] Pressure: 82.6/76.5 PSI, Flow: 38.6 gal/min
[118] Pressure: 80.9/77.7 PSI, Flow: 41.4 gal/min
[119] Pressure: 80.7/77.3 PSI, Flow: 38.7 gal/min
[120] Pressure: 83.0/76.1 PSI, Flow: 38.6 gal/min
[121] Pressure: 81.9/77.3 PSI, Flow: 39.7 gal/min
[122] Pressure: 80.7/77.8 PSI, Flow: 38.3 gal/min
[123] Pressure: 81.0/75.4 PSI, Flow: 37.1 gal/min
[124] Pressure: 81.3/74.3 PSI, Flow: 40.6 gal/min
[125] Pressure: 82.2/75.8 PSI, Flow: 40.0 gal/min
[126] Pressure: 80.6/77.7 PSI, Flow: 42.1 gal/min
[127] Pressure: 81.1/76.9 PSI, Flow: 38.8 gal/min
[128] Pressure: 82.0/74.6 PSI, Flow: 42.3 gal/min
[129] Pressure: 81.3/75.4 PSI, Flow: 42.5 gal/min
[130] Pressure: 81.0/74.6 PSI, Flow: 40.0 gal/min
[131] Pressure: 80.5/76.2 PSI, Flow: 37.3 gal/min
[132] Pressure: 80.4/75.9 PSI, Flow: 37.5 gal/min
[133] Pressure: 81.5/74.5 PSI, Flow: 39.3 gal/min
[134] Pressure: 81.4/76.2 PSI, Flow: 42.9 gal/min
[135] Pressure: 80.3/75.7 PSI, Flow: 39.8 gal/min
```

Terminal Window (Hydroficient-project — zsh — 80x24):

```
Last login: Tue Sep  8 20:29:38 on ttys000
christinekh@christinekh-MacBook-Pro ~ % cd "/Users/christinekh/Documents/IoT Hydroficient-Extern/Week3/Hydroficient-project"
christinekh@christinekh-MacBook-Pro ~ % mosquitto -v
178889253: Info: running mosquitto as user: christinekh.
178889253: mosquitto version 2.1.2 starting
178889253: Using default config.
178889253: Bridge support available.
178889253: Persistence support available.
178889253: TLS support available.
178889253: TLS-PSK support available.
178889253: Websockets support available.
178889253: Starting in local only mode. Connections will only be possible from clients running on this machine.
178889253: Create a configuration file which defines a listener to allow remote access.
178889253: For more details see https://mosquitto.org/documentation/authentication-methods/
178889253: Opening ipv4 listen socket on port 1883.
178889253: Error: Address already in use
178889253: Opening ipv6 listen socket on port 1883.
178889253: Error: Address already in use
178889253: mosquitto version 2.1.2 terminating
christinekh@christinekh-MacBook-Pro ~ % Hydroficient-project %
```

Web Dashboard (Hydroficient-project — Python dashboard_subscriber.py — 80x24):

Pressure: 81.5 / 74.5 PSI
Flow: 39.3 gal/min

Location: main-building
Device ID: GM-HYDROLOGIC-01
Timestamp: 2026-09-08T17:43:57.835805+00:00
Counter: 134
Status: NORMAL

Pressure: 81.4 / 76.2 PSI
Flow: 42.9 gal/min

Location: main-building
Device ID: GM-HYDROLOGIC-01
Timestamp: 2026-09-08T17:43:59.835874+00:00
Counter: 135
Status: NORMAL

Pressure: 80.3 / 75.7 PSI
Flow: 39.8 gal/min

Part 2: Vulnerability Analysis

Vulnerability	What's the Risk?	Evidence from My Pipeline	Potential Attack
---------------	------------------	---------------------------	------------------

No encryption	All MQTT data is transmitted in plaintext. Anyone able to observe the MQTT traffic can read the sensor information.	I intercepted complete JSON messages containing the device ID, location, timestamp, pressure readings, and flow rate without any decryption.	An attacker with access to the network could passively monitor sensor readings and learn normal water-system behavior.
No authentication	The MQTT broker does not require clients to prove their identity before connecting.	My publisher and subscriber connected to the Mosquitto broker without a username, password, or client certificate.	An unauthorized client could connect to the broker and attempt to subscribe to sensitive MQTT topics.
No authorization	Clients are not restricted to specific MQTT topics or actions.	I was able to subscribe to the wildcard topic # and receive the sensor messages from the pipeline.	An attacker who can connect to the broker could attempt to access topics they should not have access to or publish unauthorized messages.
No message verification	The system does not cryptographically verify that a message came from a trusted sensor or that it has not been modified.	The sensor messages contain a device ID and counter, but there is no HMAC, digital signature, or other integrity mechanism proving the message's origin.	An attacker could attempt to inject fake sensor readings or modify readings to make false data appear to come from the legitimate sensor.

Part 3: Remediation Recommendations

1. What defenses would I add?

1. TLS Encryption

- Encrypt MQTT traffic between sensors, the broker, and the dashboard.
- Prevent intercepted messages from being read in plaintext.
- Configure Mosquitto to require secure TLS connections.

2. Client Authentication

- Require clients to authenticate before connecting to the MQTT broker.
- Use usernames/passwords or, preferably, client certificates.
- Revoke credentials or certificates if a device is compromised.

3. Topic-Based Access Control Lists (ACLs)

- Restrict which devices can publish and subscribe to specific MQTT topics.
- Allow each sensor to publish only to its assigned topic.
- Prevent dashboards or sensors from performing actions they do not need.

4. Message Validation and Integrity Protection

- Validate timestamps to detect old or replayed messages.
- Track counters to detect duplicate or out-of-sequence messages.
- Consider using HMAC message signing to verify message integrity and authenticity.

2. What's most urgent to fix?

Priority Ranking:

Rank	Vulnerability	Why This Priority
1	No authentication	Without authentication, unauthorized clients may be able to connect to the MQTT broker. Establishing client identity is a foundational security control.
2	No encryption	Sensor data is transmitted in plaintext and can be observed by anyone able to access the MQTT traffic.
3	No authorization	After identifying clients, access should be restricted so each client has only the permissions it needs.
4	No message verification	Message integrity and freshness provide additional protection against spoofing, tampering, and replay attacks.

3. What trade-offs might these defenses have?

Defense	Trade-off
---------	-----------

TLS Encryption	Performance: Encryption adds some CPU overhead, slight latency, and certificate-management complexity.
Client Authentication	Complexity: Devices require credentials or certificates that must be securely provisioned, rotated, and revoked.
Access Control Lists	Maintenance: ACLs require ongoing maintenance when devices, topics, or permissions change.
Message Validation	Clock Sync: Timestamp and counter validation require additional processing and reliable time synchronization. HMAC also requires secure key management.